

Report - Version 8: Attention

Introduction

Version 8 replaces the gated recurrent processing from Version 7 with explicit scaled dot-product attention.

The corpus, vocabulary, four-character context windows, learned character embeddings, training and validation split, shuffled mini-batches, cross-entropy objective, automatic differentiation, and autoregressive sampling remain in place. Recurrence is removed, trainable positional embeddings preserve sequence order, and query, key, and value projections let context positions exchange information directly.

```
Characters
↓
Integer identifiers
↓
4-character context
↓
Character + positional embeddings
↓
Query, key and value projections
↓
Scaled dot-product attention
↓
Final contextual representation
↓
Output weights
↓
Generated text
```

Goal of Version 8

The model should be able to:

- reuse the same 440-character corpus, 27-character vocabulary, and four-character context windows from Version 7;
- reuse trainable 8-dimensional character embeddings and preserve the same next-character prediction task;
- add trainable positional embeddings so the four context positions remain distinguishable without recurrence;
- build scaled dot-product attention explicitly from query, key, and value projection matrices;
- preserve mini-batch training, validation monitoring, and reproducible autoregressive generation;
- verify attention shapes, normalized attention weights, gradients, parameter updates, deterministic retraining, and generation.

Architecture

The complete Version 8 flow is:

```
Corpus: 440 characters
↓
Vocabulary: 27 characters
↓
436 four-character context-target examples
↓
Integer input tensor: (436, 4)
↓
Character embeddings: (27, 8) + positions: (4, 8)
↓
Q / K / V representations: 4 x 16 per example
↓
Attention matrix: 4 x 4 per example
↓
Final contextual representation: 16 values
↓
Output weight matrix: (16, 27)
↓
Reproducible 80 / 20 split + mini-batch updates
↓
Sliding-window autoregressive generator
```

The context still contains four previous characters. Character and positional embeddings form the attention input, and the contextual representation of the final position predicts the next character.

Context windows, split and mini-batches

Sliding windows create the same 436 input-target examples as Version 7. Each example contains four ordered character identifiers; the target character is outside the input window.

```
training examples = 348
validation examples = 88
batch size = 32
updates per epoch = 11
```

Only training examples are shuffled and updated. Validation examples measure loss without modifying the model. Neighboring windows still overlap, retaining the same intentional small-dataset limitation as the previous versions.

Trainable embeddings and attention projections

Each character identifier selects one row from the trainable character embedding matrix. A trainable positional vector is added at each of the four context positions. The resulting representations are projected into queries, keys, and values before scaled dot-product attention is applied.

```
Input tensor shape: (436, 4)
Character embedding matrix shape: (27, 8)
Positional embedding matrix shape: (4, 8)
Embedded / positioned context: (1, 4, 8)
Query weight matrix shape: (8, 16)
Key weight matrix shape: (8, 16)
Value weight matrix shape: (8, 16)
Queries / keys / values: (1, 4, 16)
Attention weights shape: (1, 4, 4)
Attention output shape: (1, 4, 16)
Output weight matrix shape: (16, 27)
Logits shape: (1, 27)
Trainable parameters: 1064
```

The character embedding matrix contains 216 values and the positional embedding matrix contains 32. The query, key, and value projection matrices contain 128 values each, and the output matrix contains 432. The model therefore has 1064 trainable parameters, compared with 1800 in Version 7.

```
Q = X W_q, K = X W_k, V = X W_v
scores = Q K^T / sqrt(d_k)
attention = softmax(scores)
H = attention V
```

Stable TensorFlow loss

The context identifiers are mapped to character embeddings and combined with positional embeddings. Attention produces contextual representations, and the final position is multiplied by the output weights to produce next-character logits. Sparse softmax cross-entropy measures the prediction error directly from the logits and integer target IDs.

```
_, _, logits = attention_forward(..., inputs)
example_losses = tf.nn.sparse_softmax_cross_entropy_with_logits(
    labels=targets, logits=logits
)
loss = tf.reduce_mean(example_losses)
```

Initial train and validation loss is about 3.2958, consistent with an almost uniform 27-character prediction.

Controlled execution and reproducibility

The notebook keeps CPU execution and suppresses low-level TensorFlow logs. Explicit seeds control parameter initialization, data splitting, mini-batch shuffling, and text sampling. Repeated runs are deterministic within the same software environment.

Automatic differentiation

For each mini-batch, `tf.GradientTape` records the attention forward calculations and derives the loss with respect to all six trainable parameter matrices:

- `tf.GradientTape` records embedding lookup, positional addition, query-key-value projections, attention scores, scaling, softmax, weighted value combinations, output projection, and loss;
- `tape.gradient` calculates gradients for the character embeddings, positional embeddings, three projection matrices, and output weights;
- the sparse character-embedding gradient is converted to a dense tensor for this small vocabulary;
- `assign_sub` applies gradient descent updates to all six variables;
- training and validation loss are recorded from the current parameters at each epoch.

```
with tf.GradientTape() as tape:
    batch_loss = calculate_loss(...)
    gradients = tape.gradient(batch_loss, trainable_variables)
```

Training configuration:

```
Learning rate: 1.0
Batch size: 32
Epochs: 100
Seed: 42
```

Training result

```
Initial train loss: 3.295835
Final train loss: 2.735393
Initial validation loss: 3.295837
Final validation loss: 2.927145
Best validation loss: 2.884859
Best validation epoch: 74
```

The model changes very little during the first 20 epochs, then reduces the loss more clearly. Training remains somewhat oscillatory with the same fixed learning rate, but the overall training loss decreases substantially by epoch 100.

Validation is best at epoch 74. Later, training loss falls while validation rises slightly, suggesting mild overfitting. Generation still uses the final epoch-100 parameters.

Attention and parameter checks

The tests verify finite gradients for all six trainable parameter matrices, matching gradient shapes, finite contextual outputs, normalized attention weights, and that every trainable parameter matrix actually changes during training.

```
0 <= attention_weights <= 1
sum(attention_weights, axis=-1) = 1
assert all(gradient is not None for gradient in gradients)
assert np.max(abs(initial - trained)) > 0.0
```

Learned probabilities

For the context mode, the highest learned next-character probabilities are:

```
space: 0.0897   e: 0.0879   a: 0.0698
n: 0.0610      s: 0.0588   t: 0.0581
```

Inspecting attention weights

Attention exposes a 4 x 4 matrix whose rows sum to 1. For mode, the final row used by the output layer is:

```
m: 0.2422   o: 0.2643   d: 0.1634   e: 0.3301
final row sum: 0.99999994
```

Neural text generation

For every prediction, the current four-character window is processed by attention, the final contextual representation produces logits, and a local NumPy generator samples the next character before the window advances.

```
mode -> sampled character
ode? -> next sampled character
...
```

With seed 42, five samples from mode are r, i, t, p, and comma. The demonstration then generates 300 reproducible characters.

Included checks

Assertions verify context construction, tensor and variable types, parameter shapes, attention ranges and row sums, data splits, losses, probabilities, gradients, parameter changes, deterministic retraining, and reproducible generation.

A complete top-to-bottom run ends with the message All checks passed.

What Version 8 teaches

The central progression is:

context -> IDs -> character + positional embeddings -> Q / K / V
-> scaled attention -> contextual representation -> logits -> loss
-> gradients -> update -> slide -> sample

Version 8 replaces recurrent information flow with direct attention-based exchange: learned queries and keys compare context positions, and value vectors carry the information that is combined.

Interpretation

Trainable positional embeddings preserve order explicitly, and the same query, key, and value projections are reused at every position. Contextual outputs are computed for all four positions for inspection, while next-character prediction uses only the final position.

The model has 1064 parameters versus 1800 in Version 7. Its best validation loss is 2.8849 at epoch 74, slightly below Version 7's 2.8995; on this small corpus, the comparison is illustrative rather than evidence of general superiority.

Limitations

- the context still has a fixed length of four characters and the corpus remains very small;
- overlapping context windows can appear across the random training and validation split;
- the model intentionally uses no bias terms and no causal mask; the target character remains outside the input context;
- only the final contextual representation contributes directly to next-character prediction loss;
- there is one attention computation and no residual, normalization, feed-forward, or multi-head structure;
- training uses fixed-rate mini-batch gradient descent with no early stopping or checkpoint restoration;
- generated text captures local character patterns but still has no long-term coherence.

Conclusion

Version 8 replaces gated recurrence with explicit scaled dot-product attention. Positional embeddings preserve order, query-key comparisons produce normalized attention weights, and weighted values form contextual representations for next-character prediction.

The implementation stays inspectable. Version 9 adds causal self-attention, feed-forward processing, residual connections, and normalization.